

Table of Contents

Comprehensive Question Bank: Unit 1 — Introduction to Software Development	4
Section A: Multiple-Choice Questions (MCQs)	4
Topic 1.1: Software Development	4
Topic 1.2: Introduction to the Software Development Life Cycle (SDLC)	4
Topic 1.2.1: Framework in Software Development	5
Topic 1.2.2: Stages Involved in SDLC (Overview)	6
Topic 1.2.2.1: Requirement Gathering	6
Topic 1.2.2.2: Design	8
Topic 1.2.2.3: Coding / Development	8
Topic 1.2.2.4: Testing (SDLC Stage)	9
Topic 1.2.2.5: Deployment	9
Topic 1.2.2.6: Maintenance	10
Topic 1.3: Software Development Methodologies	10
Topic 1.3.1: Introduction to Software Process Models	11
Topic 1.3.1.1: Waterfall Model	11
Topic 1.3.1.2: Agile Methodology	12
Topic 1.4: Project Planning and Management	13
Topic 1.4.1: Comprehensive Project Planning	14
Topic 1.4.2: Setting Project Timelines	14
Topic 1.4.3: Estimating Costs	15
Topic 1.4.4: Risk Assessment and Management	15
Topic 1.4.5: Execution	16
Topic 1.4.6: Quality Assurance	16
Topic 1.5: Graphical Representation of Software Systems	17
Topic 1.5.1: Introduction to UML	17
Topic 1.5.2: Types of UML Diagrams	18
Topic 1.5.2.1: Use Case Diagrams	18
Topic 1.5.2.2: Class Diagrams	18
Topic 1.5.2.3: Sequence Diagrams	19
Topic 1.5.2.4: Activity Diagrams	19
Topic 1.5.3: Using UML to Represent Software Systems	20
Topic 1.6: Introduction to Design Patterns	20
Topic 1.6.1: Commonly Used Design Patterns	21
Topic 1.6.1.1: Singleton Pattern	21

Topic 1.6.1.2: Factory Pattern	21
Topic 1.6.1.3: Observer Pattern	21
Topic 1.6.1.4: Strategy Pattern	22
Topic 1.6.2: Applications of Design Patterns in Software Design	22
Topic 1.7: Software Debugging and Testing	23
Topic 1.7.1: Debugging	23
Topic 1.7.2: Testing	23
Topic 1.7.2.1: Unit Testing	24
Topic 1.7.2.2: Integration Testing	24
Topic 1.7.2.3: System Testing	24
Topic 1.7.2.4: Acceptance Testing	24
Topic 1.8: Software Development Tools	25
Topic 1.8.1: Language Editors	25
Topic 1.8.2: Translators	25
Topic 1.8.3: Debuggers	26
Topic 1.8.4: Integrated Development Environments (IDEs)	26
Topic 1.8.5: Online and Offline Computing Platforms	27
Topic 1.8.6: Source Code Repositories	27
Section B: Short Answer Questions	28
Topic 1.1: Software Development	28
Topic 1.2: Introduction to the Software Development Life Cycle (SDLC)	28
Topic 1.2.1: Framework in Software Development	28
Topic 1.2.2: Stages Involved in SDLC	29
Topic 1.2.2.1: Requirement Gathering	29
Topic 1.2.2.2: Design	29
Topic 1.2.2.3: Coding / Development	29
Topic 1.2.2.4: Testing (SDLC Stage)	29
Topic 1.2.2.5: Deployment	29
Topic 1.2.2.6: Maintenance	30
Topic 1.3: Software Development Methodologies	30
Topic 1.3.1: Introduction to Software Process Models	30
Topic 1.3.1.1: Waterfall Model	30
Topic 1.3.1.2: Agile Methodology	30
Topic 1.4: Project Planning and Management	30
Topic 1.4.1: Comprehensive Project Planning	31
Topic 1.4.2: Setting Project Timelines	31
Topic 1.4.3: Estimating Costs	31
Topic 1.4.4: Risk Assessment and Management	31

Topic 1.4.5: Execution	31
Topic 1.4.6: Quality Assurance	31
Topic 1.5: Graphical Representation of Software Systems	31
Topic 1.5.1: Introduction to UML	31
Topic 1.5.2.1: Use Case Diagrams	32
Topic 1.5.2.2: Class Diagrams	32
Topic 1.5.2.3: Sequence Diagrams	32
Topic 1.5.2.4: Activity Diagrams	32
Topic 1.5.3: Using UML to Represent Software Systems	32
Topic 1.6: Introduction to Design Patterns	32
Topic 1.6.1.1: Singleton Pattern	33
Topic 1.6.1.2: Factory Pattern	33
Topic 1.6.1.3: Observer Pattern	33
Topic 1.6.1.4: Strategy Pattern	33
Topic 1.6.2: Applications of Design Patterns in Software Design	33
Topic 1.7: Software Debugging and Testing	33
Topic 1.7.1: Debugging	33
Topic 1.7.2: Testing	34
Topic 1.7.2.1: Unit Testing	34
Topic 1.7.2.2: Integration Testing	34
Topic 1.7.2.3: System Testing	34
Topic 1.7.2.4: Acceptance Testing	34
Topic 1.8: Software Development Tools	34
Topic 1.8.1: Language Editors	34
Topic 1.8.2: Translators	34
Topic 1.8.3: Debuggers	34
Topic 1.8.4: Integrated Development Environments (IDEs)	35
Topic 1.8.5: Online and Offline Computing Platforms	35
Topic 1.8.6: Source Code Repositories	35

Comprehensive Question Bank: Unit 1 — Introduction to Software Development

Section A: Multiple-Choice Questions (MCQs)

Topic 1.1: Software Development

1. What is software development best defined as?
A) Writing code only, with no further steps
B) The process of creating computer programs designed to perform specific tasks
C) The process of manufacturing computer hardware
D) The process of marketing a finished application
 2. Which of the following is NOT part of the software development process as defined in this unit?
A) Writing code
B) Testing the code
C) Addressing issues that arise
D) Manufacturing physical circuit boards
 3. Why is understanding the full software development process considered important?
A) It guarantees a project will never have bugs
B) It helps create reliable, maintainable, and scalable software solutions
C) It eliminates the need for testing
D) It removes the need for a development team
 4. A common misconception about software development is that it is only:
A) Writing code
B) Planning a project
C) Testing a product
D) Maintaining a system
-

Topic 1.2: Introduction to the Software Development Life Cycle (SDLC)

1. What does the acronym SDLC stand for?
A) System Design Life Cycle
B) Software Development Life Cycle
C) Software Design Logical Cycle
D) System Development Logic Chain

2. What is the primary purpose of the SDLC?
 - A) To design the visual appearance of software only
 - B) To deliver high-quality software that meets customer expectations, on time and within cost estimates**
 - C) To eliminate the need for a development team
 - D) To manage a company's hardware inventory
 3. SDLC is best described as:
 - A) A single programming language
 - B) A framework defining the processes used to build an application from conception to deployment and maintenance**
 - C) A type of computer hardware
 - D) A marketing strategy for released software
 4. Which real-world case is commonly cited to illustrate the dangers of skipping critical SDLC stages such as requirement analysis and testing?
 - A) The invention of the Django framework
 - B) The Therac-25 radiation therapy machine incidents**
 - C) The creation of Git by Linus Torvalds
 - D) The 2001 Agile Manifesto meeting
 5. Skipping stages of the SDLC in critical systems can lead to:
 - A) Faster project completion with no downsides
 - B) Severe consequences, including catastrophic system failures**
 - C) Improved software quality
 - D) Reduced need for documentation
-

Topic 1.2.1: Framework in Software Development

1. In software engineering, a framework is best described as:
 - A) A single line of code
 - B) A standardized and reusable set of concepts, practices, and tools providing a structured foundation for software development**
 - C) A type of computer hardware
 - D) A document listing project costs
2. What is a key benefit of using a framework during software development?
 - A) It eliminates the need for testing
 - B) It allows developers to focus on application-specific code instead of reinventing common solutions**
 - C) It guarantees zero bugs
 - D) It replaces the need for requirement gathering

3. Which of the following is an example of a framework mentioned in the unit?

- A) GitHub
- B) Django**
- C) Notepad++
- D) GDB

4. Frameworks primarily promote which of the following?

- A) Slower development and inconsistency
 - B) Efficiency, consistency, and code reusability**
 - C) Elimination of the design phase
 - D) Manual hardware configuration
-

Topic 1.2.2: Stages Involved in SDLC (Overview)

1. How many major stages make up the SDLC as described in this unit?

- A) Four
- B) Five
- C) Six**
- D) Eight

2. Which of the following correctly lists the SDLC stages in order?

- A) Design, Requirements, Testing, Coding, Deployment, Maintenance
- B) Requirements, Design, Coding/Development, Testing, Deployment, Maintenance**
- C) Coding, Requirements, Design, Deployment, Testing, Maintenance
- D) Maintenance, Requirements, Design, Coding, Testing, Deployment

3. The SDLC stages are best compared to which real-world process in this unit's analogy?

- A) Cooking a meal
 - B) Constructing a multi-story building**
 - C) Painting a portrait
 - D) Writing a novel
-

Topic 1.2.2.1: Requirement Gathering

1. What is the primary goal of the requirement gathering phase?

- A) To write the final code for the software
- B) To understand and collect what the software needs to achieve**
- C) To test the software for bugs
- D) To install the software on user systems

2. Which of the following is a key activity used during requirement gathering?
- A) Writing unit tests
 - B) Conducting interviews and surveys with potential users**
 - C) Deploying the software to production
 - D) Creating a Singleton class
3. Watching how users interact with current systems to identify problems is known as:
- A) Document review
 - B) Observation**
 - C) Deployment
 - D) Debugging
4. Functional requirements primarily describe:
- A) What the system should do**
 - B) How well the system should perform
 - C) The cost of building the system
 - D) The programming language used
5. Non-functional requirements primarily describe:
- A) What the system should do
 - B) How the system should perform, including quality attributes and constraints**
 - C) Who the system's actors are
 - D) The exact source code of the system
6. "The system should allow users to register and create an account" is an example of a:
- A) Functional requirement**
 - B) Non-functional requirement
 - C) Design pattern
 - D) UML diagram
7. "The system should be available 99.9% of the time" is an example of a:
- A) Functional requirement
 - B) Non-functional requirement**
 - C) Use case
 - D) Class diagram
8. Which of the following is an example of a non-functional requirement for a Library Management System?
- A) Users can search for books
 - B) Librarians can add new books
 - C) The system should handle up to 1000 simultaneous users without performance degradation**
 - D) Users can borrow books

9. Reviewing existing documents, such as reports and user manuals, to gather additional requirement information is called:
- A) Interviews
 - B) Document review**
 - C) Acceptance testing
 - D) Risk assessment
-

Topic 1.2.2.2: Design

1. What is the main purpose of the design phase in the SDLC?
 - A) To install the software on user devices
 - B) To plan how the software will look and work based on gathered requirements**
 - C) To fix bugs found during testing
 - D) To estimate the total project budget
 2. In the design phase, diagrams are created primarily to:
 - A) Advertise the software to customers
 - B) Show how different parts of the software will connect and work together**
 - C) Replace the need for coding
 - D) Record customer complaints
 3. The design phase is compared to which activity in the unit's analogy?
 - A) Laying bricks
 - B) Drawing blueprints for a house before construction**
 - C) Handing over keys to a new homeowner
 - D) Repairing a leaking roof
 4. Which of the following activities is part of the design phase?
 - A) Running acceptance tests
 - B) Planning the software's architecture and how components will interact**
 - C) Installing the software on a server
 - D) Merging code using Git
-

Topic 1.2.2.3: Coding / Development

1. During the coding/development phase, what do programmers primarily do?
 - A) Interview stakeholders about their needs
 - B) Translate design specifications into a programming language**
 - C) Install the finished software for end users
 - D) Draw UML diagrams only

2. The coding phase directly depends on outputs from which earlier phase?
- A) Deployment
 - B) Design**
 - C) Maintenance
 - D) Acceptance Testing
-

Topic 1.2.2.4: Testing (SDLC Stage)

1. What is the primary purpose of the testing phase in the SDLC?
- A) To gather requirements from users
 - B) To check software to identify bugs, errors, or issues before release**
 - C) To install the software on the user's device
 - D) To design the software's architecture
2. Checking whether all features of the software work according to specifications is called:
- A) Functionality testing**
 - B) Performance testing
 - C) Compatibility testing
 - D) Risk assessment
3. Checking if software performs well under conditions such as high traffic is called:
- A) Functionality testing
 - B) Performance testing**
 - C) Compatibility testing
 - D) Requirement gathering
4. Ensuring software works correctly across various devices and operating systems is called:
- A) Functionality testing
 - B) Performance testing
 - C) Compatibility testing**
 - D) Document review
-

Topic 1.2.2.5: Deployment

1. Deployment in the SDLC is best defined as:
- A) Writing the initial requirements document
 - B) The process of making software available for users to access and use**
 - C) Debugging the source code
 - D) Estimating the total project cost

2. Which of the following is a step involved in deployment?
 - A) Requirement gathering
 - B) Installation of the software on the user's system or server**
 - C) Drawing a class diagram
 - D) Writing unit tests
 3. Adjusting software to fit the specific needs of a user or organization, such as setting up network settings, is called:
 - A) Installation
 - B) Configuration**
 - C) Maintenance
 - D) Acceptance testing
-

Topic 1.2.2.6: Maintenance

1. The maintenance phase of the SDLC primarily involves:
 - A) Writing the very first lines of code for a project
 - B) Ongoing updates and fixes to keep software functioning correctly over time**
 - C) Interviewing stakeholders for the first time
 - D) Drawing the system's original blueprint
 2. Why is maintenance necessary even after software works perfectly at release?
 - A) Because software never actually gets released
 - B) Because user needs and technology continue to change over time**
 - C) Because testing is skipped during release
 - D) Because deployment always fails initially
-

Topic 1.3: Software Development Methodologies

1. Software development methodologies are best defined as:
 - A) Specific programming languages used to write code
 - B) Structured approaches that guide the planning, creation, and management of software projects**
 - C) Types of computer hardware
 - D) Categories of UML diagrams
2. What historic event in 2001 is credited with revolutionizing software development methodology?
 - A) The publication of the Gang of Four's design patterns book

- B) The meeting in Snowbird, Utah, that produced the Agile Manifesto**
 - C) The creation of Git by Linus Torvalds
 - D) The invention of the Waterfall Model
-

Topic 1.3.1: Introduction to Software Process Models

1. Software process models are best described as:
 - A) Specific software products sold to customers
 - B) Abstract representations of the processes involved in the SDLC**
 - C) Types of design patterns
 - D) A category of UML diagrams
 2. Which of the following is NOT listed as a benefit of using software process models?
 - A) Predictability
 - B) Efficiency
 - C) Quality
 - D) Elimination of all project costs**
-

Topic 1.3.1.1: Waterfall Model

1. The Waterfall Model is best described as:
 - A) A flexible, iterative approach that adapts to change
 - B) A linear and sequential approach where each phase must be completed before the next begins**
 - C) A design pattern for creating single instances of objects
 - D) A type of UML diagram
2. Who is credited with originally describing the sequential Waterfall approach in 1970?
 - A) Linus Torvalds
 - B) Winston Royce**
 - C) Grady Booch
 - D) Erich Gamma
3. Which of the following is a benefit of the Waterfall Model?
 - A) High flexibility to change requirements mid-project
 - B) It is simple and easy to understand due to its clear, distinct phases**
 - C) It works best for projects with constantly evolving requirements
 - D) It allows returning to any previous phase at any time

4. Which of the following is a limitation of the Waterfall Model?
- A) It is too flexible for small projects
 - B) Once a phase is completed, going back to make changes is difficult and costly**
 - C) It does not follow a sequential process
 - D) It cannot be used for any type of project
5. The Waterfall Model is most suitable for projects where:
- A) Requirements are expected to change daily
 - B) Requirements are clear, fixed, and unlikely to change**
 - C) There is no need for a design phase
 - D) Testing can be skipped
6. Why is the term "Waterfall" used to describe this model?
- A) Because it involves working with water-related software
 - B) Because progress flows in one direction, like water flowing downward, without going back**
 - C) Because it was named after its inventor's last name
 - D) Because it requires cloud computing platforms
-

Topic 1.3.1.2: Agile Methodology

1. Agile Methodology is best described as:
- A) A rigid, sequential approach with no room for change
 - B) A flexible and adaptive approach that delivers small, functional parts of software quickly**
 - C) A tool used only for debugging code
 - D) A type of source code repository
2. In Agile, short work cycles used to deliver parts of software rapidly are called:
- A) Waterfalls
 - B) Sprints or iterations**
 - C) Deployments
 - D) Repositories
3. Regularly merging code changes into a central repository to detect and fix issues early is known as:
- A) Continuous Integration**
 - B) Test-Driven Development
 - C) Pair Programming
 - D) The Singleton Pattern

4. Writing tests before writing the actual code is a practice known as:
- A) Continuous Integration
 - B) Test-Driven Development**
 - C) Pair Programming
 - D) Acceptance Testing
5. Two developers working together at one workstation, with one writing code and the other reviewing it, describes:
- A) Continuous Integration
 - B) Test-Driven Development
 - C) Pair Programming**
 - D) Risk Assessment
6. Which of the following is a benefit of Agile Methodology?
- A) It offers a highly predictable, fixed final timeline
 - B) It allows for changes in requirements even after development has started**
 - C) It eliminates the need for stakeholder involvement
 - D) It works only for small, single-developer projects
7. Which of the following is a limitation of Agile Methodology?
- A) It cannot deliver working software quickly
 - B) Managing large projects with many teams can be difficult due to coordination challenges**
 - C) It never allows customer feedback
 - D) It requires no stakeholder participation at all
8. A client needs a mobile app in three weeks, but the requirements will change daily based on real-time data. Which methodology would generally be more suitable?
- A) Waterfall Model
 - B) Agile Methodology**
 - C) Neither model is applicable to software
 - D) Singleton Pattern
-

Topic 1.4: Project Planning and Management

1. Planning a software project is compared to which everyday activity in this unit?
- A) Cooking a recipe
 - B) Planning a trip**
 - C) Painting a house
 - D) Playing a video game

2. According to the "Did You Know?" fact in this unit, how much was Microsoft worth in 2023?
- A) \$200 Billion
 - B) \$1 Trillion
 - C) \$2 Trillion**
 - D) \$500 Billion
3. Which of the following correctly lists the 5 phases of a project management plan as described in the unit?
- A) Design, Coding, Testing, Deployment, Maintenance
 - B) Initiation, Planning, Execution, Performance Monitoring, Project Closure**
 - C) Requirements, Waterfall, Agile, Testing, Closure
 - D) Planning, Coding, Debugging, Testing, Release
-

Topic 1.4.1: Comprehensive Project Planning

1. Comprehensive project planning primarily involves:
- A) Writing the software's source code
 - B) Thinking about all the details of a project, including what needs to be done, who will do it, and how**
 - C) Testing the final product
 - D) Deploying the finished software
-

Topic 1.4.2: Setting Project Timelines

1. Setting project timelines is best described as:
- A) Estimating the total financial cost of a project
 - B) Deciding how long each part of the project will take to keep it on track**
 - C) Identifying risks that could affect the project
 - D) Writing functional requirements
2. What is a key benefit of setting project timelines?
- A) It eliminates the need for testing
 - B) It helps ensure the project stays on track and is completed on time**
 - C) It replaces the need for a design phase
 - D) It guarantees zero bugs
-

Topic 1.4.3: Estimating Costs

1. Which of the following is listed as a key factor in software project cost estimation?
A) The color scheme of the user interface
B) The development team's size, expertise, and hourly rates
C) The number of actors in a use case diagram
D) The type of UML diagram used
 2. Why does project duration typically affect cost estimation?
A) Shorter projects always cost more
B) Longer projects generally incur higher costs due to prolonged resource engagement and potential scope changes
C) Duration has no effect on cost
D) Only the technology stack affects cost
 3. Which of the following is NOT listed as a key factor in cost estimation?
A) Development Team
B) Technology Stack
C) Quality Assurance
D) The number of design patterns used in the code
-

Topic 1.4.4: Risk Assessment and Management

1. What is the first step in risk assessment and management according to this unit?
A) Develop mitigation strategies
B) Identify risks
C) Monitor and review
D) Analyze risks
2. Evaluating the likelihood of a risk occurring and its potential impact on the project is called:
A) Identifying risks
B) Analyzing risks
C) Developing mitigation strategies
D) Project closure
3. Creating a plan to reduce the likelihood or minimize the impact of a significant risk is called:
A) Identifying risks
B) Analyzing risks
C) Developing mitigation strategies
D) Execution

4. Continuously monitoring a project for new risks and adjusting strategies as necessary is called:
- A) Identifying risks
 - B) Analyzing risks
 - C) Developing mitigation strategies
 - D) Monitor and review**
5. Which of the following is an example of an external risk?
- A) A programming language changing
 - B) A team member being unavailable
 - C) Market fluctuations**
 - D) A software bug
-

Topic 1.4.5: Execution

1. The execution phase of project management is best described as:
- A) The phase where risks are only identified, not addressed
 - B) The phase where the team writes code, creates designs, and builds the software based on the plan**
 - C) The phase where the project is closed and archived
 - D) The phase where only cost estimation occurs
2. Execution primarily requires which of the following?
- A) Isolated, individual work with no communication
 - B) Teamwork, coordination, and regular updates to stay on track**
 - C) Skipping the project plan entirely
 - D) Avoiding stakeholder communication
-

Topic 1.4.6: Quality Assurance

1. Quality assurance (QA) is best defined as:
- A) The process of writing the initial source code
 - B) Ensuring a project meets set standards and works correctly**
 - C) The process of designing UML diagrams only
 - D) The process of estimating project costs
2. Which of the following is a method used in quality assurance?
- A) Ignoring stakeholder feedback
 - B) Testing, reviewing code, and getting feedback from stakeholders**
 - C) Skipping progress checks
 - D) Avoiding regular project reviews

Topic 1.5: Graphical Representation of Software Systems

1. Graphical representation of software systems primarily involves:
 - A) Writing detailed textual reports only
 - B) Using visual diagrams to depict a system's structure and behavior**
 - C) Installing software on user devices
 - D) Estimating software development costs
 2. What is a key benefit of using graphical representations for software systems?
 - A) They eliminate the need for any documentation
 - B) They simplify complex systems, making them easier to understand and communicate**
 - C) They replace the need for testing
 - D) They are only useful for non-technical stakeholders
-

Topic 1.5.1: Introduction to UML

1. What does UML stand for?
 - A) Universal Modeling Logic
 - B) Unified Modeling Language**
 - C) Unified Machine Language
 - D) Universal Markup Language
 2. Who are the "Three Amigos" credited with merging competing notations to create UML in 1994?
 - A) Erich Gamma, Richard Helm, Ralph Johnson
 - B) Grady Booch, Ivar Jacobson, and James Rumbaugh**
 - C) Linus Torvalds, Winston Royce, John Vlissides
 - D) James Rumbaugh, Erich Gamma, Linus Torvalds
 3. UML is best described as:
 - A) A programming language for writing executable code
 - B) A standardized way to visualize the design of a software system**
 - C) A type of software testing
 - D) A version control system
-

Topic 1.5.2: Types of UML Diagrams

Topic 1.5.2.1: Use Case Diagrams

1. Use Case Diagrams primarily represent a system's functionality from which perspective?
A) The developer's perspective
B) The user's perspective
C) The database's perspective
D) The hardware's perspective
 2. In a Use Case Diagram, a user who interacts with the system is called a(n):
A) Class
B) Actor
C) Method
D) Attribute
 3. Which of the following is NOT listed as a purpose of Use Case Diagrams?
A) Capturing functional requirements
B) Understanding user interactions
C) Planning and testing
D) Estimating exact project cost in currency
 4. In the Library System example, which of the following is an actor?
A) Borrow Book
B) Librarian
C) Return Book
D) Use Case
 5. What shape is typically used to represent a use case within a Use Case Diagram?
A) A rectangle
B) A stick figure
C) An oval
D) A diamond
-

Topic 1.5.2.2: Class Diagrams

1. A Class Diagram is best described as:
A) A diagram showing the order of messages exchanged over time
B) A diagram showing how things are organized in a system, including classes, attributes, and methods
C) A diagram showing only decision points in a process
D) A diagram showing actors and use cases only

2. In the "organizing your room" analogy, what does a "Box" represent in a Class Diagram?
- A) An actor
 - B) A class**
 - C) A use case
 - D) A sequence of messages
3. In a Class Diagram, actions such as `open()` or `close()` that a class can perform are called:
- A) Attributes
 - B) Methods**
 - C) Actors
 - D) Use cases
4. In a Class Diagram, data held by a class, such as "contents" or "label," is referred to as:
- A) Attributes**
 - B) Methods
 - C) Actors
 - D) Sequences
-

Topic 1.5.2.3: Sequence Diagrams

1. Sequence Diagrams primarily show:
- A) The overall structure and organization of classes
 - B) How objects in a system interact with each other in a particular sequence over time**
 - C) The decision points within a process
 - D) Who the actors of a system are
2. Sequence Diagrams are especially useful for understanding:
- A) The cost of a software project
 - B) The flow of messages between objects over time**
 - C) The attributes of a class
 - D) The risk factors of a project
-

Topic 1.5.2.4: Activity Diagrams

1. Activity Diagrams are primarily used to illustrate:
- A) The classes and attributes of a system
 - B) The flow of activities or steps in a process, including decision points**
 - C) The actors who interact with a system
 - D) The messages exchanged between two objects only

2. In the restaurant example Activity Diagram, what happens if the decision "Food Ready?" is answered "No"?
- A) The process ends immediately
 - B) The food is re-prepared**
 - C) The order is delivered
 - D) A new order is placed
-

Topic 1.5.3: Using UML to Represent Software Systems

1. According to the unit, UML diagrams can be used during which of the following stages of software development?
- A) Only during coding
 - B) Planning, development, and communication**
 - C) Only during deployment
 - D) Only during maintenance
2. UML diagrams help non-technical stakeholders primarily by:
- A) Teaching them to write source code
 - B) Helping them understand how the system works through visual communication**
 - C) Allowing them to skip requirement gathering
 - D) Replacing the need for a project manager
-

Topic 1.6: Introduction to Design Patterns

1. Design patterns are best described as:
- A) Strict rules that must never be adapted
 - B) Common, reusable solutions to problems that occur repeatedly in software development**
 - C) A type of software testing
 - D) A category of UML diagrams
2. Who are the "Gang of Four" credited with publishing 23 reusable design pattern solutions in 1994?
- A) Grady Booch, Ivar Jacobson, James Rumbaugh, Winston Royce
 - B) Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides**
 - C) Linus Torvalds and three collaborators
 - D) The creators of the Agile Manifesto
-

Topic 1.6.1: Commonly Used Design Patterns

Topic 1.6.1.1: Singleton Pattern

1. The Singleton Design Pattern ensures that:
 - A) Multiple instances of an object are created for efficiency
 - B) A specific object or resource is created only once and reused whenever needed**
 - C) Objects are created using a factory workshop
 - D) Subscribers are automatically notified of updates
 2. Which scenario best illustrates a good use case for the Singleton Pattern?
 - A) Creating a new payment method for every transaction
 - B) Ensuring an application uses only one shared database connection**
 - C) Notifying all subscribers when new content is posted
 - D) Choosing between different sorting algorithms at runtime
-

Topic 1.6.1.2: Factory Pattern

1. The Factory Design Pattern is best compared to:
 - A) A toolbox with interchangeable tools for different jobs
 - B) A workshop that creates different products without the customer needing to know how they are made**
 - C) A group of subscribers waiting for updates
 - D) A single shared object used throughout a program
 2. What is the primary benefit of using the Factory Pattern?
 - A) It guarantees only one instance of an object exists
 - B) It hides the details of object creation behind a simple request**
 - C) It automatically notifies observers of changes
 - D) It eliminates the need for object creation altogether
-

Topic 1.6.1.3: Observer Pattern

1. The Observer Design Pattern is best illustrated by which real-world example?
 - A) Ordering a pizza from a kitchen
 - B) Using a single shared database connection
 - C) Subscribing to a YouTube channel and being notified of new videos**
 - D) Selecting a payment method at checkout

2. In the Observer Pattern, the object that automatically notifies others of changes is called the:
- A) Observer
 - B) Subject**
 - C) Factory
 - D) Strategy
-

Topic 1.6.1.4: Strategy Pattern

1. The Strategy Design Pattern is best compared to:
- A) A single object shared across an entire program
 - B) A toolbox full of different tools, each designed for a specific job**
 - C) A workshop that manufactures a single type of product
 - D) A notification system for subscribers
2. What is the main advantage of using the Strategy Pattern?
- A) It ensures only one instance of a class exists
 - B) It lets you swap between interchangeable approaches to a task without rewriting surrounding code**
 - C) It automatically notifies all observers of a change
 - D) It hides object creation behind a factory
-

Topic 1.6.2: Applications of Design Patterns in Software Design

1. Which of the following is NOT listed as a benefit of using design patterns?
- A) Reducing code complexity
 - B) Enhancing code reusability
 - C) Improving communication among developers
 - D) Eliminating the need for testing entirely**
2. According to the "Did You Know?" fact, which design pattern is used in frameworks like Ruby on Rails and Angular?
- A) Singleton Pattern
 - B) Factory Pattern
 - C) Model-View-Controller (MVC) Pattern**
 - D) Observer Pattern only
-

Topic 1.7: Software Debugging and Testing

1. Debugging and testing are important primarily because they:
 - A) Guarantee software will never have bugs again
 - B) Help find and fix errors so software meets requirements and works as expected**
 - C) Replace the need for requirement gathering
 - D) Eliminate the need for deployment
-

Topic 1.7.1: Debugging

1. Debugging is best defined as:
 - A) The process of gathering user requirements
 - B) The process of finding and fixing bugs or errors in software**
 - C) The process of installing software for end users
 - D) The process of estimating project costs
 2. Which of the following is a tool or best practice for debugging?
 - A) Risk assessment
 - B) Using a debugger to step through code and inspect variables**
 - C) Writing a Requirement Specification Document
 - D) Drawing a Use Case Diagram
 3. Adding statements in code to display variable values at different points in a program is known as using:
 - A) Debuggers only
 - B) Print statements**
 - C) Code repositories
 - D) Use case diagrams
 4. Having other developers review your code to spot potential errors is called:
 - A) Debugging with print statements
 - B) Code reviews**
 - C) Acceptance testing
 - D) Risk mitigation
-

Topic 1.7.2: Testing

1. According to this unit, the testing process typically follows a hierarchy that:
 - A) Starts with the entire system and ends with individual components
 - B) Begins with smaller components and gradually progresses to the entire system**
 - C) Skips individual component testing entirely
 - D) Only occurs once, right before deployment

Topic 1.7.2.1: Unit Testing

1. Unit Testing is best described as:
 - A) Testing the interaction between multiple modules
 - B) Testing individual components or modules of software in isolation**
 - C) Testing the entire system as a whole
 - D) Testing performed by end-users before release
2. What is the primary goal of Unit Testing?
 - A) To evaluate the system's overall security
 - B) To verify that each component works correctly according to its design**
 - C) To gather user requirements
 - D) To assess project risk

Topic 1.7.2.2: Integration Testing

1. Integration Testing is performed primarily to:
 - A) Test a single function in isolation
 - B) Evaluate the interaction between different components or modules when combined**
 - C) Determine if the software is ready for release by end-users
 - D) Estimate the total project budget
2. Which of the following issues is Integration Testing specifically designed to catch?
 - A) Errors within a single isolated function
 - B) Interface errors and data flow problems between modules**
 - C) User dissatisfaction with the final product
 - D) Incorrect project cost estimates

Topic 1.7.2.3: System Testing

1. System Testing evaluates:
 - A) Only a single isolated unit of code
 - B) Only the interaction between two components
 - C) The entire software system as a whole, including functionality, performance, and security**
 - D) Only the requirements-gathering documents

Topic 1.7.2.4: Acceptance Testing

1. Acceptance Testing is primarily conducted to determine:
 - A) Whether individual functions work correctly in isolation
 - B) Whether the software is ready for release, often evaluated by end-users or clients**
 - C) Whether the project stayed within budget
 - D) Whether a design pattern was used correctly

2. Acceptance Testing is also commonly known as:

- A) System Integration Testing
 - B) User Acceptance Testing (UAT)**
 - C) Unit Verification Testing
 - D) Functional Debugging
-

Topic 1.8: Software Development Tools

1. Software development tools are best described as:

- A) Physical hardware components used to run software
- B) Programs or applications that assist in writing, editing, testing, debugging, and managing code**
- C) Marketing tools used to advertise finished software
- D) Documents used only during requirement gathering

Topic 1.8.1: Language Editors

1. Language editors, also known as code editors, are tools that primarily help developers:

- A) Estimate project costs
- B) Write and edit code in different programming languages**
- C) Manage version control history
- D) Conduct acceptance testing

2. Which of the following is an example of a language editor mentioned in the unit?

- A) GDB
- B) VS Code**
- C) GitHub
- D) PyCharm's debugger

Topic 1.8.2: Translators

1. Translators are tools that primarily:

- A) Test software for bugs
- B) Convert code written in one programming language into another language the computer can understand**
- C) Track changes to source code over time
- D) Draw UML diagrams automatically

2. A translator that converts code line-by-line is called a(n):

- A) Interpreter**
- B) Compiler
- C) Debugger
- D) Repository

3. A translator that converts the entire code at once is called a(n):
- A) Interpreter
 - B) Compiler**
 - C) IDE
 - D) Framework
4. Which of the following is an example of a compiler mentioned in the unit?
- A) The Python interpreter
 - B) GCC for C/C++**
 - C) Visual Studio Debugger
 - D) GitHub

Topic 1.8.3: Debuggers

1. The main purpose of a debugger is to help developers:
- A) Write requirement specification documents
 - B) Find and fix errors (bugs) in their code**
 - C) Estimate project timelines
 - D) Manage source code repositories
2. Which of the following is an example of a debugger mentioned in the unit?
- A) VS Code
 - B) GDB (GNU Debugger)**
 - C) Repl.it
 - D) GitHub

Topic 1.8.4: Integrated Development Environments (IDEs)

1. An IDE is best described as:
- A) A single tool used only for writing requirement documents
 - B) A comprehensive software suite integrating editors, compilers, debuggers, and version control in one place**
 - C) A type of UML diagram
 - D) A design pattern used in object-oriented programming
2. Which of the following is an IDE commonly preferred for Python development, according to the unit?
- A) Visual Studio
 - B) PyCharm**
 - C) Notepad++
 - D) GDB

Topic 1.8.5: Online and Offline Computing Platforms

1. Which of the following is an example of an online computing platform mentioned in the unit?
A) A local installation of an IDE
B) Repl.it
C) GDB
D) Notepad++
2. Offline computing platforms are best described as:
A) Cloud-based platforms accessible only via the internet
B) Local development environments on a computer
C) Tools used only for debugging
D) Tools used only for version control

Topic 1.8.6: Source Code Repositories

1. Source code repositories primarily help developers:
A) Design UML diagrams
B) Store, manage, and track changes to their code
C) Estimate project costs
D) Gather requirements from stakeholders
2. Who created Git, and under what circumstance, according to the unit?
A) Winston Royce, to formalize the Waterfall Model
B) Linus Torvalds, out of frustration when his team lost access to their existing version control system
C) The Gang of Four, to document design patterns
D) The Three Amigos, to unify UML notations
3. Which of the following is an example of a source code repository platform mentioned in the unit?
A) PyCharm
B) GitHub
C) GDB
D) Repl.it
4. What command is used to initialize a new Git repository?
A) `git commit`
B) `git init`
C) `git add`
D) `git log`

5. Which Git command is used to record a permanent snapshot of staged changes?
- A) `git init`
 - B) `git add`
 - C) `git commit`
 - D) `git log`
6. Which Git command displays the full history of commits made to a project?
- A) `git init`
 - B) `git add`
 - C) `git commit`
 - D) `git log`
7. Git's save-point system is best compared to which everyday analogy in this unit?
- A) A recipe for cooking a meal
 - B) A video game save-point system used before a risky "boss fight"**
 - C) A blueprint for constructing a house
 - D) A toolbox with interchangeable tools
-

Section B: Short Answer Questions

Topic 1.1: Software Development

1. Define software development in your own words.
2. Explain why software development is considered a "systematic" process rather than a random one.
3. Identify three stages that occur before a single line of code is written for a typical app.
4. Justify why understanding the full software development process matters for creating reliable software.

Topic 1.2: Introduction to the Software Development Life Cycle (SDLC)

1. Define SDLC and state its primary purpose.
2. Explain how the Therac-25 case illustrates the risks of skipping SDLC stages.
3. Predict what might happen to a software project if the requirement gathering stage were skipped entirely.
4. Compare SDLC to a "recipe," as described in this unit, and explain why this analogy is useful.

Topic 1.2.1: Framework in Software Development

1. Define what a framework is in the context of software engineering.
2. Explain, using the Django example, how frameworks save development time.

3. List three benefits that frameworks provide to software development teams.
4. Justify why a developer might choose to use a framework instead of writing every feature from scratch.

Topic 1.2.2: Stages Involved in SDLC

1. List the six stages of the SDLC in their correct order.
2. Compare the SDLC stages to the process of constructing a multi-story building.
3. Explain why each SDLC stage must generally build upon the one before it.

Topic 1.2.2.1: Requirement Gathering

1. Differentiate between functional and non-functional requirements.
2. Define the term "stakeholder" in the context of requirement gathering.
3. List three key activities used during requirement gathering.
4. Identify two functional requirements and two non-functional requirements for an online food delivery application.
5. Explain why validating requirements with stakeholders is an important final step.

Topic 1.2.2.2: Design

1. Explain the main purpose of the design phase in the SDLC.
2. Compare the design phase to the process of drawing blueprints for a house.
3. List the four main activities that occur during the design phase.
4. Predict what problems might occur if a software team skipped the design phase and began coding immediately.

Topic 1.2.2.3: Coding / Development

1. Define what occurs during the coding/development phase of the SDLC.
2. Explain how the coding phase depends on the outputs of the design phase.

Topic 1.2.2.4: Testing (SDLC Stage)

1. Define the testing phase of the SDLC.
2. Differentiate between functionality testing, performance testing, and compatibility testing.
3. Explain why testing is considered a "quality check" rather than an optional step.

Topic 1.2.2.5: Deployment

1. Define deployment in the context of the SDLC.
2. Differentiate between installation and configuration during deployment.

3. Explain why "testing in the real world" is still necessary even after a product has passed earlier testing stages.

Topic 1.2.2.6: Maintenance

1. Define the maintenance phase of the SDLC.
2. Explain why software maintenance is described as a "never-ending" stage.
3. Identify two reasons an app might require maintenance years after its initial release.

Topic 1.3: Software Development Methodologies

1. Define what a software development methodology is.
2. Explain the significance of the 2001 meeting in Snowbird, Utah, for the software industry.

Topic 1.3.1: Introduction to Software Process Models

1. Define what a software process model is.
2. List the three benefits software process models provide, according to this unit.

Topic 1.3.1.1: Waterfall Model

1. Define the Waterfall Model and explain why it is named "Waterfall."
2. List the six main phases of the Waterfall Model.
3. Compare the benefits and limitations of the Waterfall Model.
4. Identify the type of project for which the Waterfall Model is best suited, and justify your answer.
5. Explain why it is difficult to return to a previous phase once it is completed in the Waterfall Model.

Topic 1.3.1.2: Agile Methodology

1. Define Agile Methodology and explain the meaning of a "sprint" or "iteration."
2. Differentiate between Continuous Integration, Test-Driven Development, and Pair Programming.
3. Compare the benefits and limitations of Agile Methodology.
4. Given a project where requirements change daily, justify whether Waterfall or Agile would be the more appropriate methodology.
5. Explain why Agile projects are often described as "less predictable" than Waterfall projects.

Topic 1.4: Project Planning and Management

1. Define project planning and explain why it is compared to planning a trip.
2. List the five phases of a project management plan in their correct order.

Topic 1.4.1: Comprehensive Project Planning

1. Define comprehensive project planning.
2. Explain what three key questions comprehensive project planning must answer.

Topic 1.4.2: Setting Project Timelines

1. Define what it means to set project timelines.
2. Explain how setting timelines helps keep a software project on track.

Topic 1.4.3: Estimating Costs

1. Define cost estimation in the context of software project planning.
2. List five key factors that influence software project cost estimation.
3. Explain how the technology stack chosen for a project can affect its overall cost.

Topic 1.4.4: Risk Assessment and Management

1. Define risk assessment and management.
2. List and briefly explain the four steps involved in risk assessment and management.
3. Differentiate between a technical risk, an operational risk, and an external risk, providing one example of each.
4. Identify two risks that could threaten a small student software project, and propose a mitigation strategy for each.

Topic 1.4.5: Execution

1. Define the execution phase of project management.
2. Explain why teamwork and coordination are described as essential to the execution phase.

Topic 1.4.6: Quality Assurance

1. Define quality assurance in the context of software project management.
2. List the methods used to achieve quality assurance, according to this unit.

Topic 1.5: Graphical Representation of Software Systems

1. Define graphical representation of software systems.
2. Explain how visual diagrams help simplify complex software systems for stakeholders.

Topic 1.5.1: Introduction to UML

1. Define UML and state its main purpose.
2. Explain the historical significance of "The Three Amigos" in the development of UML.

Topic 1.5.2.1: Use Case Diagrams

1. Define a use case diagram and explain its purpose.
2. Differentiate between an "actor" and a "use case" within a Use Case Diagram.
3. List the four steps involved in identifying use cases.
4. Map out the actors and use cases you would expect for an online food ordering application.

Topic 1.5.2.2: Class Diagrams

1. Define a class diagram and explain what it is used to represent.
2. Differentiate between an "attribute" and a "method" within a class diagram, using the "organizing your room" example.
3. Identify the attributes and methods you would expect for a **Student** class in a school management system.

Topic 1.5.2.3: Sequence Diagrams

1. Define a sequence diagram and explain what it shows.
2. Differentiate between a Class Diagram and a Sequence Diagram in terms of what each one represents.
3. Map out, in words, the sequence of messages exchanged when a user logs into a mobile banking app.

Topic 1.5.2.4: Activity Diagrams

1. Define an activity diagram and explain what it is used to model.
2. Differentiate between a Sequence Diagram and an Activity Diagram, highlighting their key differences.
3. Map out an activity diagram, in words, for the process of logging into a website, including what happens for both correct and incorrect passwords.

Topic 1.5.3: Using UML to Represent Software Systems

1. Explain the three practical applications of UML mentioned in this unit.
2. Justify why UML diagrams are useful for communicating with non-technical stakeholders.

Topic 1.6: Introduction to Design Patterns

1. Define what a design pattern is in software development.
2. Explain the historical significance of the "Gang of Four" in the development of modern design patterns.

Topic 1.6.1.1: Singleton Pattern

1. Define the Singleton Design Pattern.
2. Explain why a banking application might want only one instance of a database connection class.

Topic 1.6.1.2: Factory Pattern

1. Define the Factory Design Pattern.
2. Describe the Factory Pattern and explain how it differs from directly creating objects, with an example.

Topic 1.6.1.3: Observer Pattern

1. Define the Observer Design Pattern.
2. Explain, using the YouTube subscription example, how the Observer Pattern keeps things "in sync."

Topic 1.6.1.4: Strategy Pattern

1. Define the Strategy Design Pattern.
2. Explain how the Strategy Pattern allows a program to switch between different approaches without rewriting surrounding code.

Topic 1.6.2: Applications of Design Patterns in Software Design

1. List three benefits that design patterns provide to software development.
2. Explain how the Model-View-Controller (MVC) pattern relates to frameworks like Ruby on Rails and Angular.

Topic 1.7: Software Debugging and Testing

1. Explain why debugging and testing are considered essential steps in software development.
2. Justify the statement: "Bugs are not a sign of failure, but a normal part of engineering."

Topic 1.7.1: Debugging

1. Define debugging and explain how it differs from testing.
2. List three tools or best practices used for debugging.
3. Explain how using breakpoints in a debugger helps a developer locate the source of a bug.

Topic 1.7.2: Testing

1. Explain why the testing process typically follows a hierarchy, starting with small components and moving toward the entire system.

Topic 1.7.2.1: Unit Testing

1. Define unit testing and explain what a "unit" refers to.
2. Explain the primary goal of unit testing.

Topic 1.7.2.2: Integration Testing

1. Define integration testing and differentiate it from unit testing.
2. Explain why two individually correct modules might still fail when combined, and how integration testing catches this.

Topic 1.7.2.3: System Testing

1. Define system testing and explain what aspects of software it evaluates.
2. Differentiate between system testing and integration testing.

Topic 1.7.2.4: Acceptance Testing

1. Define acceptance testing and explain who typically performs it.
2. Explain why acceptance testing is sometimes called "User Acceptance Testing (UAT)."

Topic 1.8: Software Development Tools

1. Define software development tools and explain their overall purpose.
2. Explain the historical circumstances under which Linus Torvalds created Git in 2005.

Topic 1.8.1: Language Editors

1. Define a language editor and give two examples mentioned in this unit.

Topic 1.8.2: Translators

1. Differentiate between an interpreter and a compiler.
2. Explain why understanding the difference between interpreters and compilers might matter when locating a bug in a long program.

Topic 1.8.3: Debuggers

1. Define a debugger and explain its role in software development.
2. Identify two examples of debuggers mentioned in this unit.

Topic 1.8.4: Integrated Development Environments (IDEs)

1. Define an IDE and explain what tools it typically integrates.
2. Explain why an IDE might be more efficient for a developer than using separate, standalone tools.

Topic 1.8.5: Online and Offline Computing Platforms

1. Differentiate between online and offline computing platforms, with an example of each.
2. Explain one advantage and one disadvantage of using an online computing platform.

Topic 1.8.6: Source Code Repositories

1. Define a source code repository and explain its role in version control.
 2. Explain, step by step, the purpose of the `git init`, `git add`, `git commit`, and `git log` commands.
 3. Compare Git's version control system to the "video game save-point" analogy used in this unit.
 4. Justify why source code repositories are especially important when multiple developers work on the same project.
-